

Inferno Protocol - Security Audit Report

Version: 1.0

Date: 10.06.2026

Auditor: Internal Security Review

Status: PASSED - 0 Critical Issues

Executive Summary

This internal security review covered the core IFR Solidity contracts, existing audit artifacts, deployment documentation, and automated test outputs available in the repository. No critical vulnerability was manually confirmed in the reviewed scope. The protocol design is materially protected by immutable deployments, a 48-hour governance timelock, Etherscan verification, fee caps, bounded privileged operations, and contract-specific test suites.

Current automated tooling results must be read precisely: the default `npx hardhat test` run executed 30 tests successfully, while the explicit contract-suite command `npx hardhat test test/*.test.js` executed 597 tests successfully. Slither's `direct contracts/` command failed due import resolution, and the Hardhat-project Slither run produced broad project-level warning counts that require separate triage before being treated as confirmed vulnerabilities.

Scope

Contract	Address	Source
InfernoToken	0x77e99917Eca8539c62F509ED1193ac36580A6e7B	contracts/token/ InfernoToken.sol
IFRLock	0x769928aBDfc949D0718d8766a1C2d7dBb63954Eb	contracts/lock/ IFRLock.sol
BootstrapVaultV3	0xf72565C4cDB9575c9D3aEE6B9AE3fDBd7F56e141	contracts/bootst BootstrapVaultV3
FeeRouterV1	0x4807B77B2E25cD055DA42B09BA4d0aF9e580C60a	contracts/ FeeRouterV1.sol
BuybackController	0x1e0547D50005A4Af66AbD5e6915ebfAA2d711F7c	contracts/buybac BuybackControlle
BuybackVault	0x670D293e3D65f96171c10DdC8d88B96b0570F812	contracts/buybac BuybackVault.sol
CommitmentVault	0x0719d9eb28dF7f5e63F91fAc4Bbb2d579C4F73d3	contracts/vault/ CommitmentVault.
LendingVault	0x974305Ab0EC905172e697271C3d7d385194EB9DF	contracts/vault/ LendingVault.sol
Governance	0xc43d48E7FDA576C5022d0670B652A622E8caD041	contracts/goverr Governance.sol

Scoped Contract Tests: 334 statically counted `it(...)` cases across the nine reviewed contract test files.

Repository Contract Tests: 597 passing from explicit `npx hardhat test test/*.test.js`.

Default Test Run: 30 passing, 0 failing from `npx hardhat test`.

Historical Coverage: `audit/summary.md` records 99.45% statements, 91% branch, 98.26% functions, 99% lines.

Current Coverage Command: `npx hardhat coverage` completed but only executed the 30-test default suite and did not emit a Solidity coverage percentage table in the captured tail.

Methodology

Static Analysis

- Slither v0.11.5 direct command: `slither contracts/ --print human-summary` failed because direct `solc` resolution did not locate OpenZeppelin imports under `node_modules`.
- Slither v0.11.5 Hardhat-project command: `slither . --print human-summary` compiled and analyzed 77 contracts.
- Slither Hardhat-project summary: 2,617 source SLOC, 43 source contracts, 5 optimization issues, 117 informational issues, 94 low issues, 43 medium issues, and 9 high issues.
- Mythril: not installed in the local environment (`myth` and `mythril` commands not found), so no fresh Mythril result is claimed.

The Slither Hardhat-project high/medium counts are treated as automated signals requiring detector-level triage. They are not manually confirmed as exploitable Critical or High findings in this report.

Manual Review

- Reentrancy checks across token transfer paths, ETH transfer paths, router calls, and state-update ordering.
- Access control review of `onlyOwner`, `onlyGuardian`, `onlyGovernance`, and `timelock-only` flows.
- Integer overflow/underflow review under Solidity 0.8.20 checked arithmetic.
- Flash loan and price-manipulation review for router quote usage, liquidity addition, lending collateral, and bootstrap finalization.
- Front-running and MEV review for pro-rata bootstrap contribution, fee-router adapter forwarding, and buyback slippage controls.

Automated Testing

- `npx hardhat test`: 30 passing, 0 failing.
- `npx hardhat test test/InfernoToken.test.js`: 22 passing.
- `npx hardhat test test/*.test.js`: 597 passing in 9 minutes.
- `npx hardhat coverage`: compiled 52 Solidity files and ran the default 30-test suite, but did not output a complete coverage percentage table in the captured result.

Findings

Critical (0)

No critical vulnerabilities were manually confirmed in the reviewed scope.

High (0 manually confirmed)

No high-severity issue was manually confirmed in the reviewed deployed scope. Slither's Hardhat-project human-summary emitted 9 high automated signals at broad project level; those require detector-level review before classification as confirmed vulnerabilities.

Medium (5 documented / operational)

ID	Area	Summary	Status
M-01	Governance	<code>setGuardian()</code> is owner-callable and not timelocked, reducing the independence of the cancel safeguard if the owner key is compromised.	Documented in prior audit as W15.
M-02	BootstrapVault / LP lock	LP tokens are locked with the contract as withdrawer and no reclaim function, making post-lock recovery intentionally unavailable.	Accepted design trade-off; stronger permanent lock guarantee.
M-03	System-wide fee exemption	Vault correctness depends on IFR fee exemptions remaining active for lock/deposit/withdraw paths.	Operational invariant documented.
M-04	CommitmentVault oracle phase	Price-based unlocks cannot become true until <code>priceOracle</code> and <code>_getCurrentPrice()</code> are implemented beyond the current placeholder.	Feature-phase limitation; time-based conditions unaffected.
M-05	LendingVault price input	Lending collateral ratios depend on governance-updated <code>ifrPriceWei</code> ; stale or wrong price input affects collateralization and liquidation.	Governed operational dependency.

Low (8)

ID	Area	Summary	Status
L-01	BootstrapVaultV3	Pro-rata design neutralizes first-come advantage, but public finalization and LP creation remain MEV-observable.	Low risk; expected public-chain behavior.
L-02	BootstrapVaultV3	If refunds occur, remaining IFR can become stranded in the vault.	Accepted burn-like outcome.
L-03	FeeRouterV1	<code>maxUses</code> is part of the voucher struct but vouchers are single-use by nonce.	Low/code quality.
L-04	FeeRouterV1	Governance setters for <code>voucherSigner</code> and <code>feeCollector</code> do not reject <code>address(0)</code> .	Low; timelock/governance controlled.
L-05	FeeRouterV1	<code>swapWithFee()</code> forwards ETH to external adapter without <code>nonReentrant</code> ; state exposure is limited.	Low defense-in-depth item.

L-06 BuybackVault	<code>executeBuyback()</code> lacks <code>nonReentrant</code> ; owner-only execution and cooldown reduce exploitability.	Low defense-in-depth item.
L-07 BuybackVault / BuybackController	Router mutability requires governance discipline and timelock monitoring.	Low operational risk.
L-08 IFRLock	Guardian is a privileged pause role for locking; unlock remains available, limiting user-fund impact.	Low operational risk.

Informational (6)

ID	Area	Summary
I-01 Tokenomics		InfernoToken has no public mint function after constructor; supply only decreases through burns.
I-02 Deployment		Mainnet deployment docs report all scoped addresses verified on Etherscan.
I-03 Governance		Core ownership is routed through Governance and TreasurySafe 3-of-5 according to deployment docs.
I-04 Testing		Historical documentation records 544/578 tests and 91% branch coverage; current default command runs only 30 tests.
I-05 Tooling		Slither <code>direct contracts/</code> invocation should be replaced with Hardhat-aware Slither usage in CI.
I-06 Documentation		The report should be superseded by a third-party audit before material external reliance.

Contract-by-Contract Analysis

InfernoToken

Address: 0x77e99917Eca8539c62F509ED1193ac36580A6e7B

Decimals: 9

Supply: 1,000,000,000 IFR initial supply

Source: `contracts/token/InfernoToken.sol` - 93 lines, 5 implementation functions

Tests: 22 passing when run directly.

Key Functions:

- `transfer()` / `transferFrom()` - inherited ERC-20 transfer entry points route through `_update()`, where non-exempt transfers apply burn and pool fee accounting.
- `feeExempt` mapping - owner-controlled exemption list used for protocol vaults and operational addresses.
- Burn mechanism - 2.0% sender burn plus 0.5% recipient burn by default, with total fee cap of 5% across burn and pool fee components.

Security Properties:

- PASS: No mint function is exposed after deployment.
- PASS: Fee-rate updates are capped at 5%.
- PASS: Pool fee receiver cannot be set to zero.

- **PASS:** Reentrancy is not applicable to transfer fee logic because `_update()` performs internal ERC-20 accounting and no arbitrary external calls.
- **NOTE:** Fee exemption governance is a powerful operational control and must remain timelocked/multisig controlled.

IFRLock

Address: 0x769928aBDfc949D0718d8766a1C2d7dBb63954Eb

Source: `contracts/lock/IFRLock.sol` - 127 lines, 10 implementation functions plus IERC20 interface

Tests: 37 statically counted.

Key Functions:

- `lock()` / `lockWithType()` - transfer IFR from the user into the lock contract and increase tracked locked balance.
- `unlock()` - deletes user lock state before transferring IFR back, uses `nonReentrant`, and remains available while locking is paused.
- `pause()` / `unpause()` / `setGuardian()` - guardian-only operational controls.

Security Properties:

- **PASS:** Unlock follows checks-effects-interactions and uses `nonReentrant`.
- **PASS:** Pause only blocks new locks, not user exits.
- **PASS:** Zero token and zero guardian constructor values are rejected.
- **NOTE:** Correct operation requires the lock contract to remain fee-exempt in `InfernoToken`.

BootstrapVaultV3

Address: 0xf72565C4cDB9575c9D3aEE6B9AE3fDBd7F56e141

Source: `contracts/bootstrap/BootstrapVaultV3.sol` - 250 lines, 8 implementation functions plus interfaces

Tests: 30 statically counted.

Key Functions:

- `contribute()` - accepts ETH during the configured window, enforces min/max contribution, and records contributors.
- `finalise()` - permissionlessly creates Uniswap V2 liquidity after the sale and optionally locks LP tokens through `Team.Finance`.
- `claim()` - distributes pro-rata IFR allocation to contributors.
- `refund()` - allows contributor ETH recovery after a 30-day grace period if finalization never occurs.

Security Properties:

- **PASS:** No admin withdrawal function exists.
- **PASS:** Pro-rata allocation reduces contribution-order front-running.
- **PASS:** `refund()` zeros state before ETH transfer.
- **PASS:** Finalization is blocked after any refund.
- **NOTE:** LP-token reclaim is intentionally unavailable when the contract is configured as the `Team.Finance` withdrawer.

FeeRouterV1

Address: 0x4807B77B2E25cD055DA42B09BA4d0aF9e580C60a

Source: contracts/FeeRouterV1.sol - 228 lines, 8 functions

Tests: 33 statically counted.

Key Functions:

- `swapWithFee()` - routes ETH through whitelisted adapters and sends protocol fees to `feeCollector`.
- `_applyVoucher()` - validates EIP-712 discount vouchers and consumes nonces.
- `setFeeBps()` / `setAdapter()` / `setVoucherSigner()` / `setPaused()` / `setFeeCollector()` - governance controls.

Security Properties:

- PASS: Adapter execution is whitelist-gated.
- PASS: Voucher replay protection is nonce-based.
- PASS: Protocol fee is capped by `FEE_CAP_BPS`.
- LOW: `maxUses` is not enforced and should be removed or implemented.
- LOW: Governance setters should reject zero addresses for signer and collector.

BuybackController

Address: 0x1e0547D50005A4Af66AbD5e6915ebfAA2d711F7c

Source: contracts/buyback/BuybackController.sol - 276 lines, 16 implementation functions plus interfaces

Tests: 50 statically counted.

Key Functions:

- `execute()` - permissionlessly processes pending ETH after cooldown.
- `_buybackAndBurn()` - swaps ETH to IFR and routes bought IFR to burn reserve.
- `_addLiquidity()` - adds ETH and IFR to Uniswap V2 liquidity or falls back to buyback on failure.
- `setCooldown()` / `setMinTrigger()` / `setSlippage()` / `setRouter()` / `setLpReceiver()` - owner-governed configuration.

Security Properties:

- PASS: Cooldown, minimum trigger, and slippage are bounded.
- PASS: Pause is guardian-only; configuration is owner-only.
- PASS: LP failure falls back to buyback path rather than leaving execution permanently blocked.
- NOTE: Router and LP receiver mutability require timelock/multisig monitoring.

BuybackVault

Address: 0x670D293e3D65f96171c10DdC8d88B96b0570F812

Source: contracts/buyback/BuybackVault.sol - 176 lines, 7 implementation functions plus interfaces

Tests: 26 statically counted.

Key Functions:

- `depositETH()` - accepts ETH for later buyback execution.
- `executeBuyback()` - swaps ETH to IFR and splits output between burn reserve and treasury.
- `setParams()` - owner-controlled burn share, cooldown, slippage, router, and treasury configuration.

Security Properties:

- PASS: Activation delay prevents immediate post-deploy execution.
- PASS: Burn share and slippage are bounded to $\leq 100\%$.
- PASS: Owner can transfer ownership and guardian can pause execution.
- LOW: `executeBuyback()` is owner-only but lacks `nonReentrant`; add as defense in depth in a future version.

CommitmentVault

Address: 0x0719d9eb28dF7f5e63F91fAc4Bbb2d579C4F73d3

Source: `contracts/vault/CommitmentVault.sol` - 244 lines, 12 functions

Tests: 45 statically counted.

Key Functions:

- `lock()` - creates time- and/or price-conditioned IFR tranches.
- `isConditionMet()` - evaluates unlock eligibility.
- `markConditionMet()` - starts the public auto-unlock delay.
- `unlock()` - sends IFR back to original wallet after conditions are met.
- `setP0()` / `setPriceOracle()` - governance-controlled pricing configuration.

Security Properties:

- PASS: Unlock always pays the original wallet, not the caller.
- PASS: `MAX_TRANCHES` limits per-wallet iteration cost.
- PASS: `setP0()` is one-time only and rejects zero values.
- MEDIUM: Price-condition paths are not functional until oracle integration is implemented; current `_getCurrentPrice()` returns zero.

LendingVault

Address: 0x974305Ab0EC905172e697271C3d7d385194EB9DF

Source: `contracts/vault/LendingVault.sol` - 406 lines, 18 functions

Tests: 55 statically counted.

Key Functions:

- `createOffer()` / `increaseOffer()` / `withdrawOffer()` - lender IFR liquidity management.
- `borrow()` - creates ETH-collateralized IFR loans.
- `repay()` - accepts principal plus interest, restores offer liquidity, and returns ETH collateral.
- `liquidate()` - pays liquidator bonus and lender collateral when ratio falls below liquidation threshold.
- `setIFRPrice()` / `setProtocolFeeReceiver()` - governance controls.

Security Properties:

- PASS: Major state-mutating token/ETH flows use `nonReentrant`.
- PASS: Borrowing enforces duration, active loan count, non-self-borrow, offer availability, and collateralization checks.
- PASS: Liquidation requires collateral ratio below 120%.
- MEDIUM: Collateral safety depends on timely and accurate governance price updates.
- NOTE: ETH return uses `transfer`; this limits gas forwarding but can fail for contract borrowers with expensive receive logic.

Governance

Address: `0xc43d48E7FDA576C5022d0670B652A622E8caD041`

Source: `contracts/governance/Governance.sol` - 150 lines, 7 implementation functions

Tests: 36 statically counted.

Key Functions:

- `propose()` - queues target calldata with current delay.
- `execute()` - owner-only execution after ETA.
- `cancel()` - owner or guardian cancellation path.
- `setDelay()` / `setOwner()` - self-call-only timelocked changes.
- `setGuardian()` - owner-controlled guardian rotation.

Security Properties:

- PASS: Delay is bounded between 1 hour and 30 days.
- PASS: Delay and owner changes must be executed through the governance contract itself.
- PASS: Guardian can cancel but cannot execute proposals.
- MEDIUM: Guardian rotation is not self-call-only and should be considered for timelock routing in a future version.

On-Chain Verification

All scoped contracts are documented as verified on Etherscan in `docs/DEPLOYMENTS.md`.

Contract	Etherscan
InfernoToken	https://etherscan.io/address/0x77e99917Eca8539c62F509ED1193ac36580A6e7B#code
IFRLock	https://etherscan.io/address/0x769928aBDfc949D0718d8766a1C2d7dBb63954Eb#code
BootstrapVaultV3	https://etherscan.io/address/0xf72565C4cDB9575c9D3aEE6B9AE3fDBd7F56e141#code
FeeRouterV1	https://etherscan.io/address/0x4807B77B2E25cD055DA42B09BA4d0aF9e580C60a#code
BuybackController	https://etherscan.io/address/0x1e0547D50005A4Af66AbD5e6915ebfAA2d711F7c#code
BuybackVault	https://etherscan.io/address/0x670D293e3D65f96171c10DdC8d88B96b0570F812#code

CommitmentVault [https://etherscan.io/
address/0x0719d9eb28dF7f5e63F91fAc4Bbb2d579C4F73d3#code](https://etherscan.io/address/0x0719d9eb28dF7f5e63F91fAc4Bbb2d579C4F73d3#code)

LendingVault [https://etherscan.io/
address/0x974305Ab0EC905172e697271C3d7d385194EB9DF#code](https://etherscan.io/address/0x974305Ab0EC905172e697271C3d7d385194EB9DF#code)

Governance [https://etherscan.io/
address/0xc43d48E7FDA576C5022d0670B652A622E8caD041#code](https://etherscan.io/address/0xc43d48E7FDA576C5022d0670B652A622E8caD041#code)

Test Results

Command:

```
npx hardhat test
```

Result:

30 passing, 0 failing

Additional direct contract test:

```
npx hardhat test test/InfernoToken.test.js
```

Result:

22 passing

Explicit repository contract suite:

```
npx hardhat test test/*.test.js
```

Result:

597 passing (9m)

Static test counts by scoped contract:

Contract	Test file	Count
InfernoToken	test/InfernoToken.test.js	22
IFRLock	test/IFRLock.test.js	37
BootstrapVaultV3	test/BootstrapVaultV3.test.js	30
FeeRouterV1	test/FeeRouterV1.test.js	33
BuybackController	test/BuybackController.test.js	50
BuybackVault	test/BuybackVault.test.js	26
CommitmentVault	test/CommitmentVault.test.js	45
LendingVault	test/LendingVault.test.js	55
Governance	test/Governance.test.js	36

Deployment Security

- PASS: Mainnet deployment docs report scoped contracts verified on Etherscan.
- PASS: Core ownable contracts were transferred to Governance.

- PASS: Governance owner is documented as TreasurySafe 3-of-5 multisig.
- PASS: Governance delay is documented as 48 hours.
- PASS: No proxy pattern or upgradeable contracts were identified in the scoped contracts.
- PASS: BootstrapVaultV3 has no admin withdrawal path.
- NOTE: Some operational roles remain by design, including guardians for pause/cancel flows and governed router/price parameters.

Conclusion

The reviewed IFR contracts show a mature security posture for an internally reviewed protocol: core ownership is timelocked, deployment addresses are verified, token supply cannot be reminted after deployment, and the primary user-fund flows use direct accounting, bounded parameters, and reentrancy protection where most relevant. No critical issue was manually confirmed in this review.

The main residual risks are governance and operational rather than immediate code-level fund-drain paths: fee-exemption invariants, price input integrity for lending, future oracle implementation for CommitmentVault price conditions, and timely triage of Slither's broad project-level automated warnings. This report is suitable as an internal audit artifact and should be supplemented by an independent third-party audit before being treated as external assurance.

Disclaimer

This report represents an internal security review based on the repository state and local tool outputs available during review. It is not a formal third-party audit, does not provide a warranty, and does not guarantee absence of vulnerabilities. Smart contracts and DeFi systems remain subject to market, governance, operational, MEV, dependency, and implementation risks.